INFORME PRÁCTICO

SISTEMAS OPERATIVOS

# LINUX USUARIOS PERMISOS

INFORME PRÁCTICO

KALI LINUX

SISTEMAS OPERATIVOS



ACCESO CONTROLADO

USUARIOSGRUPOSPERMISOSPRUEBAS

01

- USUARIOS Y GRUPOS
- PERMISOS OCTALES
- ARCHIVO PRIVADO
- PRUEBAS DE ACCESO

INFORME PRÁCTICO EN KALI LINUX: GRUPOS DEPARTAMENTALES, USUARIOS Y UN ARCHIVO PRIVADO PROTEGIDO CON PERMISOS OCTALES.

GESTIÓN DE USUARIOS, GRUPOS Y PERMISOS EN LINUX

# 01

# FUNDAMENTOS DE ACCESO

Identidad, grupos y permisos octales: lo necesario para leer la práctica.



# LA SEGURIDAD EN LINUX EMPIEZA POR LA IDENTIDAD

Linux decide qué puede hacer cada persona a partir de tres datos: quién es, a qué grupo pertenece y qué permisos tiene el recurso. Esta práctica recorre esos tres datos con un caso de tres departamentos.

1 →
### IDENTIDAD
Cada cuenta de usuario tiene un número propio, el UID.

2 →
### PERTENENCIA
Cada usuario pertenece a un grupo principal, con su GID.

3
### PERMISO
Cada archivo o directorio dice qué puede hacer su propietario, su grupo y el resto.

**PUNTO CLAVE**

El permiso no se da a personas sueltas: se da a un **grupo**, y las personas entran por pertenecer a él.

# USUARIOS, GRUPOS Y SUS IDENTIFICADORES

El sistema guarda las cuentas en `/etc/passwd` y los grupos en `/etc/group`. En la práctica, cada línea de `/etc/passwd` muestra nombre, UID, GID, directorio personal e intérprete de comandos.

| DEPARTAMENTO | GRUPO | USUARIO | UID | GID | DIRECTORIO |
|---|---|---|---|---|---|
| **Sistemas** | sistemas | ana | 1001 | 1001 | /srv/sistemas |
| **Redes** | redes | bruno | 1002 | 1002 | /srv/redes |
| **Soporte** | soporte | carla | 1003 | 1003 | /srv/soporte |

Tabla 1. Cuentas, grupos y directorios del escenario; UID y GID leídos con `id` **FUENTE**.

**3**
grupos
**FUENTE**

**3**
usuarios
**FUENTE**

**3**
directorios
**FUENTE**

**1**
archivo privado
**FUENTE**

# LEER UN PERMISO OCTAL

Cada archivo guarda tres juegos de permisos: para la persona propietaria, para el grupo propietario y para el resto. Cada juego suma lectura **r = 4**, escritura **w = 2** y ejecución **x = 1**. Un directorio necesita `x` para poder entrar en él.

| OCTAL | SÍMBOLOS | SUMA | SIGNIFICADO |
|---|---|---|---|
| **7** | rwx | 4 + 2 + 1 | leer, escribir y ejecutar o entrar |
| **6** | rw- | 4 + 2 | leer y escribir |
| **0** | --- | 0 | sin ningún permiso |

Tabla 2. Los tres dígitos que usa esta práctica CALC.

## 770

DIRECTORIOS /SRV/…

+ propietario: rwx
+ grupo: rwx
+ resto: ---

## 660

PRIVADO_SISTEMAS.TXT

+ propietario: rw-
+ grupo: rw-
+ resto: ---

**En ambos casos el resto del sistema queda sin acceso.**

# ANATOMÍA DE UNA LÍNEA DE LS -L

El comando `ls -l` resume en una línea el estado de un archivo. Esta es la línea del archivo privado, tal como aparece en la captura del Punto 3.

BASH

```
-rw-rw---- 1 root sistemas 0 oct 18 16:37 privado_sistemas.txt
```

Salida de ls -l sobre el archivo privado FUENTE.

| CAMPO | VALOR | QUÉ INDICA |
|---|---|---|
| **Tipo y permisos** | -rw-rw---- | archivo normal; permisos 660 |
| **Enlaces** | 1 | un solo nombre para el archivo |
| **Propietario** | root | cuenta que creó el archivo con `sudo` |
| **Grupo** | sistemas | grupo con acceso de lectura y escritura |
| **Tamaño** | 0 | bytes: aún vacío antes de las pruebas |
| **Fecha** | oct 18 16:37 | última modificación |
| **Nombre** | privado_sistemas.txt | archivo |

Tabla 3. Campos de la línea de `ls -l` FUENTE.

# CÓMO DECIDE EL SISTEMA SI DEJA PASAR

Al pedir acceso, el sistema compara la cuenta con el propietario y el grupo del recurso, y aplica **solo un** juego de permisos: el primero que coincide. El superusuario `root` no está sujeto a esta comprobación, por eso las pruebas usan `sudo -u` para actuar como cada cuenta. SIN FUENTE

# ELECCIÓN DEL JUEGO DE PERMISOS APLICABLE

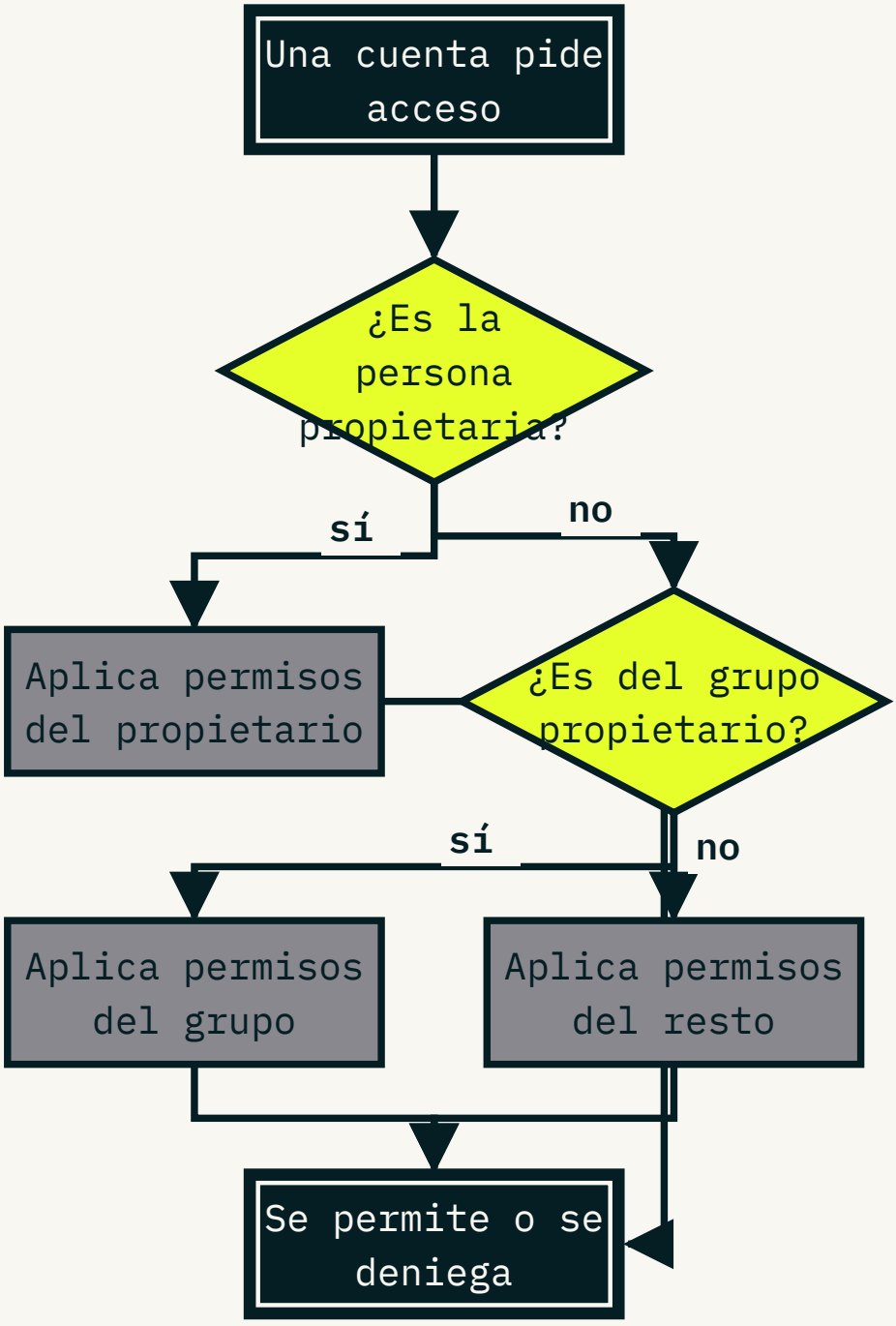



Figura 1. Regla de coincidencia del modelo de permisos POSIX SIN FUENTE.

# 02

# DESARROLLO PRÁCTICO

Los cuatro puntos de la actividad, con sus comandos y capturas.



# RUTA DE LA PRÁCTICA

La actividad avanza de lo general a lo específico: primero las cuentas, luego los directorios, después el archivo y al final la comprobación.

1 →
### 1. USUARIOS Y GRUPOS
`groupadd` y `useradd` crean tres grupos y tres cuentas.

2 →
### 2. DIRECTORIOS
`mkdir`, `chown` y `chmod 770` en `/srv/`.

3 →
### 3. ARCHIVO PRIVADO
`touch`, `chown` y `chmod 660` en `/srv/sistemas/`.

4
### 4. PRUEBAS
`sudo -u` simula el acceso de ana y de bruno.

| COMANDO | QUÉ HACE |
|---|---|
| `groupadd` | crea un grupo |
| `useradd -m -g grupo usuario` | crea el usuario con directorio personal (`-m`) y grupo principal (`-g`) |
| `mkdir -p` | crea el directorio y las rutas intermedias que falten |
| `chown :grupo ruta` | cambia solo el grupo propietario |
| `chmod modo ruta` | cambia los permisos |
| `touch` | crea un archivo vacío |
| `sudo -u usuario comando` | ejecuta el comando como otra cuenta |

Tabla 4. Comandos usados en la práctica SIN FUENTE.

# PUNTO 1: USUARIOS Y GRUPOS

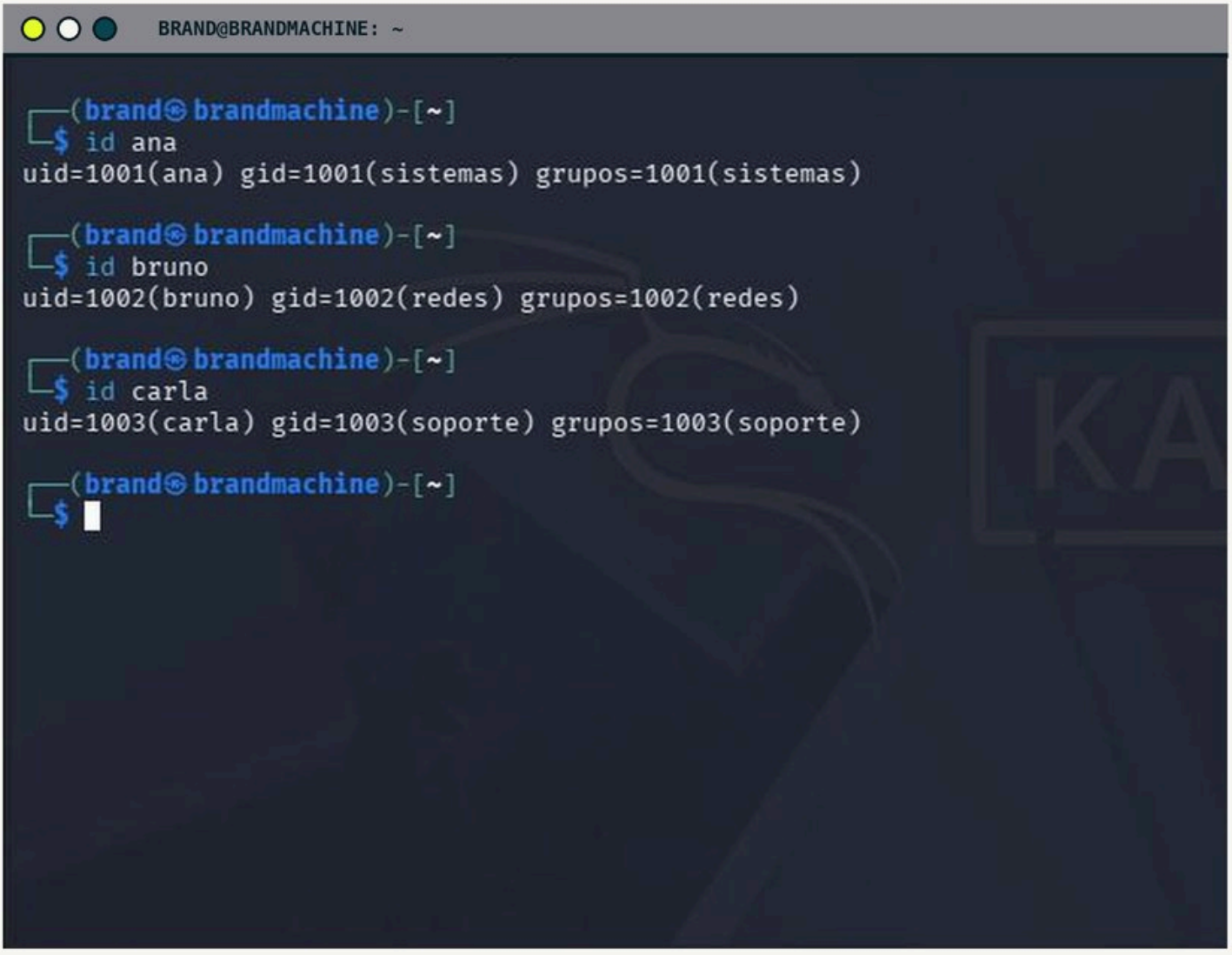
Se crearon los tres grupos departamentales y, después, un usuario por grupo con su grupo principal.

BASH

```
  sudo groupadd sistemas
  sudo groupadd redes
  sudo groupadd soporte

  sudo useradd -m -g sistemas ana
  sudo useradd -m -g redes bruno
  sudo useradd -m -g soporte carla

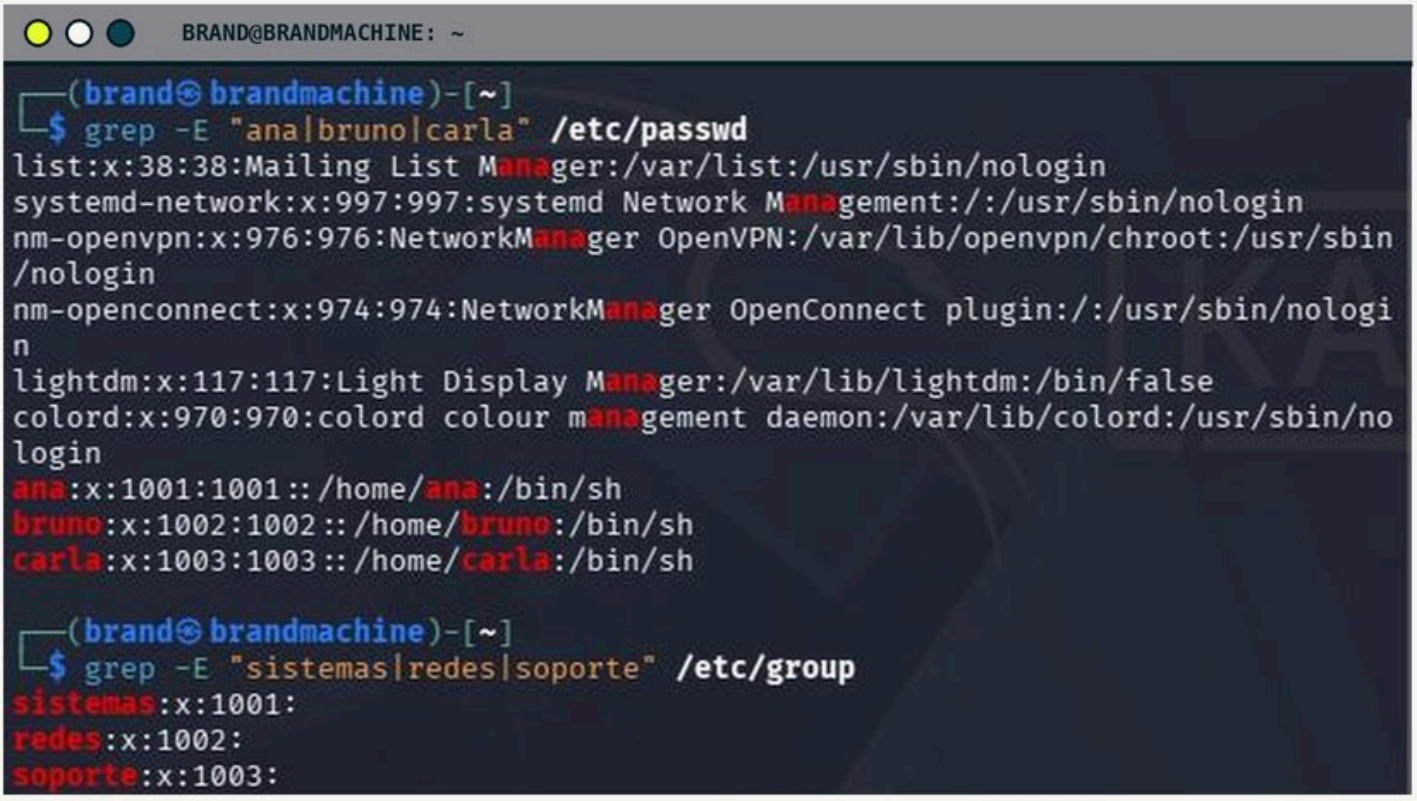
Comandos ejecutados.
```




Salida de `id` para ana, bruno y carla: cada UID y GID coincide con su grupo **FUENTE**.

# CUENTAS REGISTRADAS EN EL SISTEMA

Las líneas de ana, bruno y carla aparecen resaltadas en rojo, con `/home/<usuario>` como directorio y `/bin/sh` como intérprete de comandos.




Búsqueda con `grep -E` en `/etc/passwd` y `/etc/group` **FUENTE**.

**LECTURA DE LA CAPTURA**

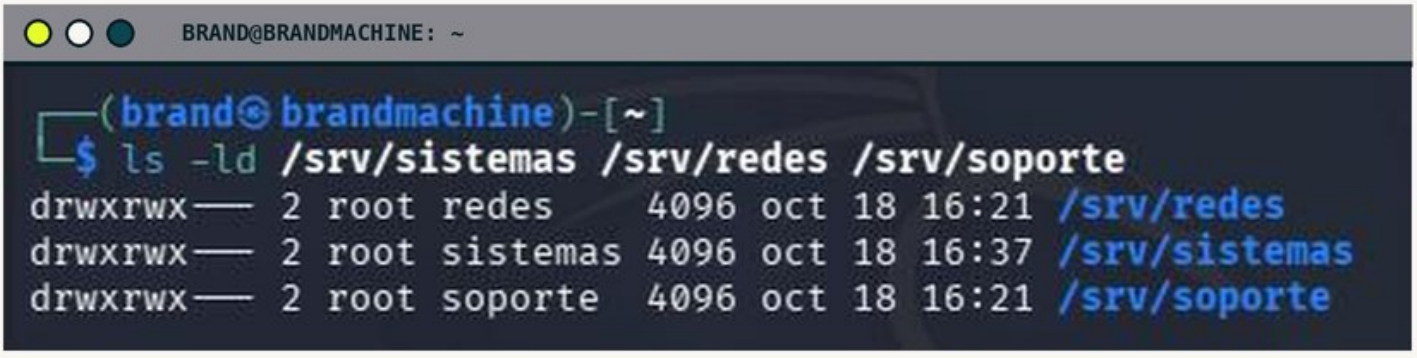
El filtro `ana|bruno|carla` también atrapa líneas ajenas (por ejemplo «Manager»), porque busca la cadena dentro de cualquier palabra. Para el informe solo cuentan las tres líneas en rojo.

# PUNTO 2: DIRECTORIOS DEPARTAMENTALES

Se creó un directorio por departamento en `/srv/`, con el grupo del departamento como propietario y modo `770`.

```bash
sudo mkdir -p /srv/sistemas /srv/redes /srv/soporte
sudo chown :sistemas /srv/sistemas
sudo chown :redes /srv/redes
sudo chown :soporte /srv/soporte
sudo chmod 770 /srv/sistemas /srv/redes /srv/soporte

Comandos ejecutados.
```




Resultado de `ls -ld`: `drwxrwx---`, propietario `root`, grupo del departamento **FUENTE**.

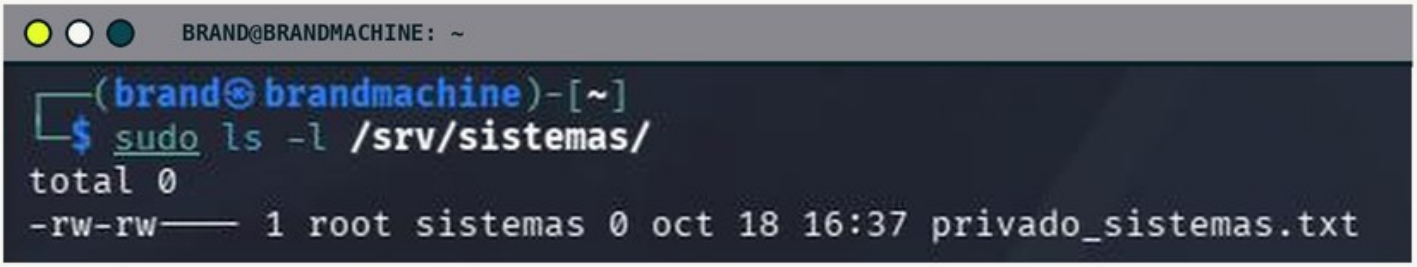
# PUNTO 3: ARCHIVO PRIVADO

Dentro del directorio de Sistemas se creó `privado_sistemas.txt`, con grupo `sistemas` y modo `660`.

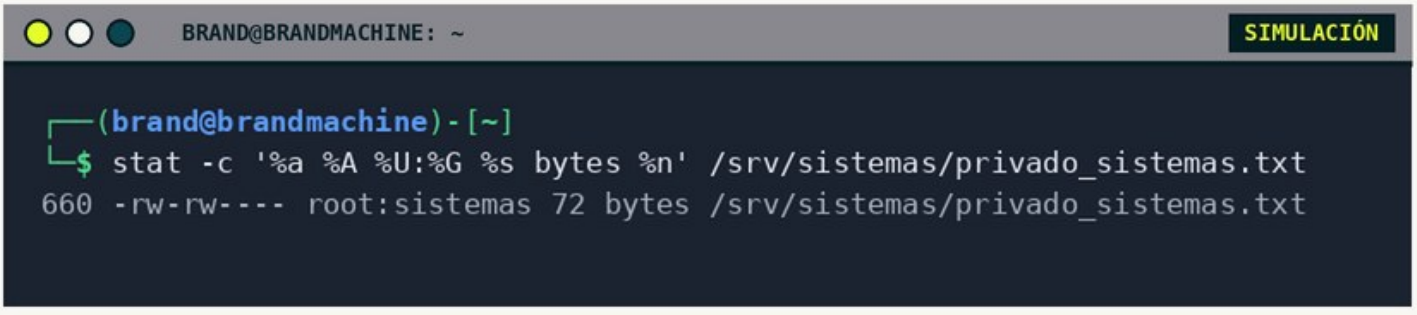
BASH

```
sudo touch /srv/sistemas/privado_sistemas.txt
sudo chown :sistemas /srv/sistemas/privado_sistemas.txt
sudo chmod 660 /srv/sistemas/privado_sistemas.txt

Comandos ejecutados.
```




`ls -l`: permisos `-rw-rw----`, tamaño 0 antes de las pruebas **FUENTE**.




Reconstrucción `SIMULACIÓN` de `stat`: modo `660`, propietario `root`, grupo `sistemas`. Los 72 bytes salen de 3 líneas de 24 bytes **CALC**.

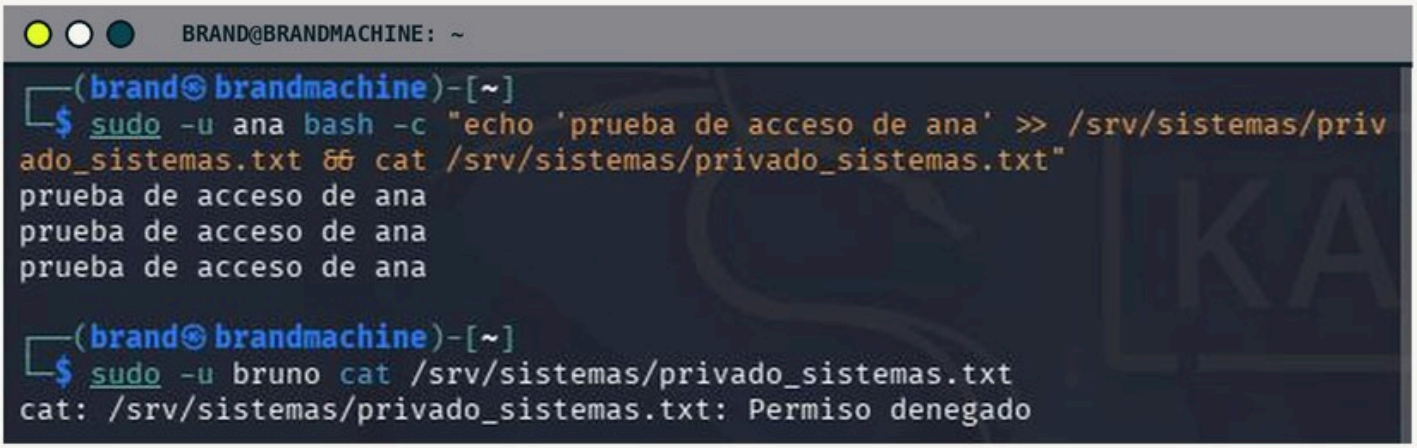
# PUNTO 4: PRUEBAS DE ACCESO

Se simuló el acceso de la usuaria autorizada (ana, del grupo sistemas) y de un usuario de otro departamento (bruno, del grupo redes) con `sudo -u`.

```bash
# Prueba con Ana (grupo sistemas)
sudo -u ana bash -c "echo 'prueba de acceso de ana' >>
/srv/sistemas/privado_sistemas.txt && cat
/srv/sistemas/privado_sistemas.txt"

# Prueba con Bruno (grupo redes)
sudo -u bruno cat /srv/sistemas/privado_sistemas.txt

Comandos ejecutados.
```



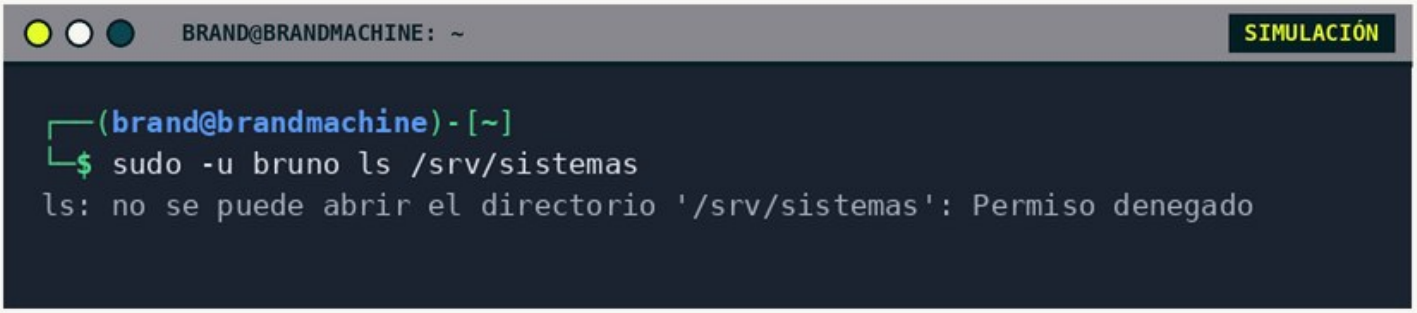

Ana escribe y lee el archivo; a Bruno el sistema le responde «Permiso denegado» FUENTE.

# PRUEBAS RECONSTRUIDAS (SIMULACIÓN)

Estas dos pruebas no se ejecutaron en la práctica. Las imágenes son reconstrucciones `SIMULACIÓN` de lo que el sistema debe responder con los permisos configurados; sustitúyelas por capturas reales cuando se ejecuten.




Reconstrucción `SIMULACIÓN`: carla, del grupo soporte, no puede leer el archivo.




Reconstrucción `SIMULACIÓN`: bruno no puede listar el directorio de Sistemas.

# 03

# VERIFICACIÓN Y ANÁLISIS

Qué se comprobó, por qué funcionó y dónde están los límites.



# MATRIZ DE VERIFICACIÓN

| QUIÉN | GRUPO | ACCIÓN | RESULTADO ESPERADO | RESULTADO OBSERVADO |
|---|---|---|---|---|
| **ana** | sistemas | escribir y leer el archivo | permitido (`rw-` de grupo) | **permitido** FUENTE |
| **bruno** | redes | leer el archivo | denegado (resto `---`) | **denegado** FUENTE |
| **carla** | soporte | leer el archivo | denegado (resto `---`) | **denegado** SIMULACIÓN |
| **bruno** | redes | entrar en /srv/sistemas | denegado (resto `---`) | **denegado** SIMULACIÓN |

Tabla 5. Lo comprobado FUENTE y lo reconstruido SIMULACIÓN.

**SIMULACIÓN**

Las filas de carla y de la entrada de bruno no se ejecutaron: muestran el resultado que corresponde a los permisos y están rotuladas SIMULACIÓN. Sustitúyelas por capturas reales.

# POR QUÉ ANA ENTRA Y BRUNO NO

El archivo tiene propietario `root` y grupo `sistemas`. Ninguna de las dos cuentas es la propietaria, así que el sistema pasa a comparar el grupo.

## ANA

GRUPO SISTEMAS

+ no es propietaria
+ su grupo coincide con el del archivo
+ aplica `rw-`
+ lee y escribe

## BRUNO

GRUPO REDES

+ no es propietario
+ su grupo no coincide
+ aplica el resto: `---`
+ recibe «Permiso denegado»

**La diferencia está en el grupo, no en la persona.**

DATO DE LA CAPTURA

Tras la prueba de ana, el contenido muestra tres líneas idénticas. Eso sugiere que la orden se ejecutó tres veces. INF

# ALCANCE Y LÍMITES DE LA PRÁCTICA

- Solo se usó el modelo básico de propietario, grupo y resto.
- Cada cuenta tiene un único grupo; no se probó una persona en varios grupos.
- La comprobación real cubre a ana y bruno; las pruebas de carla y de la entrada al directorio son reconstrucciones `SIMULACIÓN`.
- Los permisos se verificaron con `ls` y con pruebas de lectura y escritura, no con una auditoría del sistema.



La reacción típica ante un «Permiso denegado».
Crédito: **dato pendiente**

**OJO**

Un permiso correcto en el archivo no basta si el directorio que lo contiene deja pasar a quien no debe. Por eso se controlan los dos niveles.

# 04

# SEGURIDAD Y CIERRE

Buenas prácticas, siguientes pasos y conclusiones.



# BUENAS PRÁCTICAS DE ADMINISTRACIÓN

Lo hecho en la práctica ilustra principios que se aplican en cualquier servidor. Son recomendaciones generales del equipo, no resultados medidos. INF

- Dar a cada persona solo el acceso que necesita para su trabajo.
- Asignar permisos a grupos y no a cuentas sueltas, para que cambiar de área sea cambiar de grupo.
- Evitar abrir un recurso a todo el sistema solo para que «funcione».
- Comprobar siempre el resultado con `ls -l` y `id` después de cambiar permisos.
- Documentar cada comando y su salida, como en este informe.

**REGLA**

Un permiso nuevo se comprueba con una cuenta que **debe** entrar y otra que **no** debe entrar.

# SIGUIENTES PASOS DE SIMPLE A COMPLEJO

| | |
|---|---|
| 1 | **COMPLETAR LA EVIDENCIA**<br>Capturar las pruebas de carla y de la entrada al directorio.<br>`costo: bajo` |
| 2 | **VARIOS GRUPOS POR PERSONA**<br>Probar una cuenta con grupos adicionales con `usermod -aG`.<br>`costo: bajo` |
| 3 | **PERMISOS FINOS**<br>Estudiar las listas de control de acceso (ACL) con `setfacl` para casos que el modelo octal no cubre.<br>`costo: medio` |

Estos pasos son propuestas del equipo para ampliar la práctica. INF

# CONCLUSIONES

1. Las cuentas de usuario y los grupos primarios pedidos por la actividad quedaron configurados.
2. El modo `770` en los directorios deja a las personas ajenas a cada área sin permisos sobre ellos. Esa conclusión se apoya en los permisos de `ls -ld`; la entrada de otra cuenta solo tiene reconstrucción **SIMULACIÓN**.
3. Las pruebas con `sudo -u` confirmaron que ana accede al archivo de su área y que a bruno se le denegó el acceso por pertenecer al grupo redes.

> **La seguridad depende del grupo propietario y de un tercer dígito en cero: quien no es del grupo no entra.**
>
> — EQUIPO DEL INFORME

# REFERENCIAS

1 Páginas del manual de Linux: `useradd`, `groupadd`, `chown`, `chmod`, `mkdir`, `touch`, `sudo`. Versión y fecha de consulta: **dato pendiente**.

2 Capturas de pantalla y salidas de terminal de la ejecución del equipo en Kali Linux, 18 de octubre de 2026.

# CRÉDITOS DE IMÁGENES

Las capturas reales son del equipo. Las fotos esperan su crédito.

| IMAGEN | AUTOR | LICENCIA | ORIGEN |
|---|---|---|---|
| **Capturas de terminal (5)** | Equipo del informe | propia | Kali Linux, 18 de octubre de 2026 |
| **Terminales reconstruidas (3)** | Equipo del informe | propia | Generadas con `servidor/tools/terminal_sim.py` SIMULACIÓN |
| **Foto de portada** | **dato pendiente** | **dato pendiente** | **dato pendiente** |
| **Foto de divisores** | **dato pendiente** | **dato pendiente** | **dato pendiente** |
| **Foto «Permiso denegado»** | **dato pendiente** | **dato pendiente** | **dato pendiente** |

Tabla 6. Créditos de las imágenes del documento.